ĐẠI HỌC KINH TẾ QUỐC DÂN

BÁO CÁO BÀI TẬP NHÓM

Học phần: Lập trình hướng đối tượng

HỆ THỐNG QUẢN LÝ BÃI ĐỖ XE SINH VIÊN

Thiết kế và cài đặt bằng C++

Giảng viên hướng dẫn: Nguyễn Ngọc Tuấn

Lớp học phần: CNTT1131(126)_01

Nhóm thực hiện: Nhóm 9

STT	Họ và tên sinh viên	Mã sinh viên
1	Nguyễn Minh Hiếu	11232111
2	Nguyễn Trung Hiếu	11253091
3	Vũ Nhật Quang	11244024

Hà Nội, năm 2026

Tóm tắt

Một lượt gửi xe trong trường đại học thường bắt đầu bằng việc giao thẻ và kết thúc bằng việc đối chiếu xe, thu tiền, nhận lại thẻ. Các bước này ngắn nhưng có liên hệ chặt chẽ: thẻ phải gắn với đúng xe, mức phí phụ thuộc thời điểm vào và ra, còn lịch sử cần được giữ lại sau khi thẻ đã chuyển sang lượt khác. Đề tài lựa chọn quy trình đó để xây dựng một chương trình quản lý bãi xe bằng C++.

Chương trình phục vụ nhân viên bãi xe, quản lý xe máy và xe máy điện qua giao diện console. Thao tác quẹt thẻ được mô phỏng bằng nhập mã; biển số được nhập thủ công. Ba mức phí 3.000, 5.000 và 7.000 đồng được xác định theo mốc 19:00 trong cùng ngày. Lượt qua ngày được chuyển sang thu phí thủ công kèm lý do. Khi sinh viên làm mất thẻ, chương trình tra theo biển số, lưu thông tin sinh viên/chủ xe và thu phí thủ công. Các giao dịch thành công được lưu vào tệp để có thể tiếp tục quản lý sau khi mở lại chương trình.

Thiết kế tách xe, lượt gửi và bãi xe khỏi giao diện và lớp lưu trữ. Các quy tắc phí cùng kế thừa một lớp trừu tượng, cho phép bộ tính phí gọi qua giao diện chung. Ngoại lệ được phân loại theo lỗi nhập liệu, xung đột, sức chứa và lưu trữ. Khi thay đổi dữ liệu, dịch vụ chuẩn bị trạng thái dự kiến, lưu tệp rồi mới áp dụng vào bộ nhớ.

Kết quả kiểm tra gồm 45 ca C++ và sáu kịch bản console đạt yêu cầu. Báo cáo dành phần chính để giải thích các quyết định thiết kế, theo dõi những tình huống cụ thể và đánh giá giới hạn của cách lưu tệp. Sản phẩm phù hợp phạm vi bài tập nhóm, vận hành một máy và chưa tích hợp thiết bị ở cổng bãi xe.

Từ khóa: C++; lập trình hướng đối tượng; quản lý bãi xe; đa hình; xử lý ngoại lệ.

Mục lục

1 Mở đầu **1**

1.1 Bài toán được lựa chọn 1

1.2 Mục tiêu của đề tài 1

1.3 Phạm vi và các giả định 2

1.4 Cách tiếp cận và cấu trúc báo cáo 3

2 Yêu cầu phần mềm và nghiệp vụ **4**

2.1 Người sử dụng và các nhóm chức năng 4

2.2 Nhận xe: dữ liệu và điều kiện chấp nhận 5

2.3 Trả xe và thời điểm ghi nhận thu tiền 6

2.4 Xử lý mất thẻ 7

2.5 Biểu phí và các trường hợp biên 8

2.6 Tra cứu, thống kê và chất lượng dữ liệu 9

3 Thiết kế lớp và đối tượng **10**

3.1 Từ quy trình gửi xe đến mô hình lớp 10

3.2 Quan hệ giữa các lớp 11

3.3 Định danh: mã thẻ không phải mã lượt 11

3.4 Trạng thái của lượt và đối tượng báo giá 12

3.5 Ràng buộc và giao diện nghiệp vụ 13

4 Vận dụng lập trình hướng đối tượng **15**

4.1 Đóng gói: giữ đối tượng ở trạng thái hợp lệ 15

4.2 Kế thừa tại các quy tắc tính phí 17

4.3 Đa hình trong một lần tính tiền 18

4.4 Ngoại lệ và đường đi của lỗi 19

4.5 Quản lý tài nguyên và tính nhất quán 21

5 Cài đặt và xử lý dữ liệu **22**

5.1 Tổ chức chương trình và giao diện 22

5.2 Luồng xử lý nhận xe 23

5.3 Từ tra thẻ đến xác nhận trả xe 23

5.4 Tệp dữ liệu và cách khôi phục 25
5.5 Ngày giờ, tiền và thống kê 26
5.6 Build và chạy chương trình 27

6 Kiểm thử và đánh giá 28
6.1 Cách chọn tình huống kiểm tra 28
6.2 Biên 19:00 và tính đầy đủ của biểu phí 28
6.3 Hủy trả xe và sử dụng lại thẻ 29
6.4 Mất thẻ và thông tin sinh viên/chủ xe 30
6.5 Lỗi ghi tệp và khôi phục dữ liệu 30
6.6 Kết quả chạy qua giao diện console 31
6.7 Đánh giá thiết kế và giới hạn sử dụng 32

7 Kết luận và hướng phát triển 33
7.1 Mức đáp ứng mục tiêu 33
7.2 Những lựa chọn cần cân nhắc khi mở rộng 33
7.3 Hướng phát triển 33

A Ma trận kiểm thử và dữ liệu minh họa 35
A.1 Kết quả kiểm thử C++ 35
A.2 Dữ liệu kết thúc kịch bản console 36

Danh mục hình và bảng

Danh mục hình

2.1 Các nhóm thao tác của nhân viên trong phạm vi hệ thống 4
2.2 Quy trình trả xe với hai bước xác nhận của nhân viên 7
2.3 Ba lượt mẫu có cùng mốc phân chia 19:00 8
3.1 Sơ đồ các quan hệ chính; thuộc tính và phương thức được rút gọn 11
3.2 Trạng thái lưu trữ của một lượt gửi 12
4.1 Các lớp con cùng cung cấp giao diện kiểm tra và tính phí 17
5.1 Luồng nhận xe; chỉ bước cuối thay trạng thái đang chạy 23
5.2 Trình tự rút gọn của một lượt trả xe cùng ngày 24
5.3 Các bước thay thế tệp dữ liệu và vị trí bản sao lưu 26

Danh mục bảng

1.1 Mục tiêu học phần và phần cài đặt dùng để minh họa 2
2.1 Danh sách yêu cầu chức năng 5
2.2 Đặc tả thao tác nhận xe 6
2.3 Điều kiện lựa chọn mức phí cho xe máy và xe máy điện 8
3.1 Các lớp chính và phần thông tin do lớp quản lý 10
3.2 Ví dụ thẻ được dùng lại trong các lượt độc lập 12
3.3 Giao diện nghiệp vụ được giao diện console sử dụng 13
4.1 Theo dõi các lời gọi cho lượt 18:00 đến 19:00 19
4.2 Phân loại lỗi trong mô hình và lưu trữ 19
5.1 Các điểm vào của giao diện console 22
5.2 Các trường của một lượt trong tệp dữ liệu 25
6.1 Phân nhóm 45 ca kiểm thử C++ 28
6.2 Các ca ranh giới và số tiền mong đợi 29
6.3 Trạng thái mong đợi trong các nhánh trả xe 30
6.4 Đối chiếu trước và sau lỗi ghi khi trả xe 31
6.5 Sáu kịch bản console đã chạy 31
A.1 Ma trận đầy đủ của 45 ca kiểm thử 35
A.2 Các lượt được tạo trong kịch bản C01 37

1. Mở đầu

1.1 Bài toán được lựa chọn

Một sinh viên vào bãi, nhận thẻ rồi đến lớp. Khi ra về, sinh viên đưa lại thẻ, nhân viên kiểm tra xe và thu tiền. Nếu chỉ nhìn vào các thao tác tại cổng, quy trình khá đơn giản. Tuy nhiên, phía sau mỗi lần giao thẻ là một số thông tin cần đi cùng nhau: xe nào đang gửi, dùng thẻ nào, vào lúc nào và đã thanh toán hay chưa. Chỉ cần một thông tin bị ghi nhận sai, các bước sau sẽ khó kiểm tra lại.

Đề tài đặt bài toán trong bối cảnh bãi xe sinh viên của Đại học Kinh tế Quốc dân, với đối tượng quản lý chính là xe máy và xe máy điện. Những quy tắc cụ thể về giá vé, mốc thời gian và thao tác nhân viên được giới hạn theo yêu cầu của bài tập. Đây là mô hình phần mềm phục vụ học phần, chưa phải kết quả khảo sát hoặc hệ thống được triển khai tại bãi xe của trường.

Hai đặc điểm của bài toán ảnh hưởng trực tiếp đến thiết kế. Thứ nhất, một thẻ được dùng lại nhiều lần, trong khi lịch sử của từng lần gửi phải được giữ riêng. Thứ hai, mức phí không chỉ phụ thuộc vào thời điểm xe vào. Một xe vào trước 19:00 có thể trả 3.000 đồng nếu ra trước mốc này, nhưng phải trả 7.000 đồng nếu ra từ 19:00 cùng ngày. Chương trình cần ghi lại cả thời gian vào lẫn thời gian ra để quyết định đúng mức tiền. Một tình huống khác là sinh viên làm mất thẻ: khi đó không còn mã thẻ để tra cứu, nên nhân viên phải tìm lượt theo biển số, thu phí thủ công và ghi lại thông tin sinh viên cùng chủ xe trên giấy tờ.

Một vấn đề khác xuất hiện khi nhân viên đang trả xe nhưng chưa xác nhận thu tiền. Màn hình có thể đã hiển thị mức phí, song xe vẫn chưa hoàn tất lượt gửi. Nếu chương trình cho phép dùng lại thẻ ngay khi tra cứu, thẻ có thể được giao cho xe khác trong lúc giao dịch cũ chưa kết thúc. Vì vậy, thao tác xem phí và thao tác xác nhận trả xe phải được phân biệt ngay từ yêu cầu nghiệp vụ.

1.2 Mục tiêu của đề tài

Về sử dụng, chương trình cần giúp nhân viên theo dõi các xe còn trong bãi, nhận và trả đúng thẻ, tính tiền nhất quán và xem lại giao dịch sau khi đóng chương trình. Việc thao tác được tổ chức bằng menu console để nhóm có thể tập trung vào dữ liệu và quy trình xử lý. Các thiết bị như camera hoặc đầu đọc thẻ được thay bằng thao tác nhập, tránh để phần tích hợp phần cứng chiếm phần lớn khối lượng của bài tập.

Về học phần, mục tiêu là chỉ ra mỗi nguyên lý hướng đối tượng giải quyết việc gì trong chương trình. Việc khai báo nhiều lớp chưa đủ để giải thích một thiết kế. Báo cáo cần làm rõ vì sao có lớp lượt gửi bên cạnh lớp xe, vì sao bộ tính phí sử dụng kế thừa, và vì sao một lỗi ghi tệp phải truyền về giao diện thay vì bị bỏ qua.

Bảng 1.1. Mục tiêu học phần và phần cài đặt dùng để minh họa

Nội dung	Câu hỏi cần giải thích trong đề tài
Yêu cầu phần mềm	Nhận/trả xe thành công khi nào; các trường hợp nào phải từ chối?
Lớp và đối tượng	Xe, lượt gửi, bãi xe lưu thông tin gì và phối hợp ra sao?
Kế thừa	Các quy tắc phí có phần chung và phần riêng như thế nào?
Đa hình	Một lời gọi qua `FeeRule` chọn đúng mức phí bằng cách nào?
Ngoại lệ	Lỗi được phát hiện ở đâu, bắt ở đâu và dữ liệu còn lại thế nào?

Sản phẩm được xem là đạt mục tiêu khi thực hiện được toàn bộ một lượt gửi, dùng lại thẻ mà không mất lịch sử, mở lại chương trình vẫn thấy xe chưa ra, và xử lý được các lỗi đã nêu trong yêu cầu. Những điều kiện này cũng là cơ sở để xây dựng bộ kiểm thử thay vì chỉ kiểm tra chương trình có mở được menu hay không.

1.3 Phạm vi và các giả định

Nhân viên là người trực tiếp dùng phần mềm. Sinh viên tham gia quy trình giao nhận thẻ và thanh toán, nhưng không đăng nhập hoặc thao tác trên màn hình chương trình. Mã thẻ được nhập từ bàn phím để mô phỏng thao tác quẹt. Vì không có ảnh camera, biển số được nhập thủ công lúc nhận xe và hiển thị lại khi trả để nhân viên đối chiếu với xe thực tế.

Chương trình hiện tại dùng một máy và một phiên làm việc với cùng tệp dữ liệu. Mỗi thẻ chỉ gắn với một lượt đang gửi; một biển số cũng chỉ có một xe đang gửi tại một thời điểm. Chương trình chưa quản lý danh mục thẻ vật lý được cấp phát. Một mã thẻ hợp lệ về cú pháp và chưa được dùng trong bãi được xem là sẵn sàng cho thao tác nhận xe.

Xe máy và xe máy điện có cùng biểu phí. Phí tự động chỉ áp dụng khi giờ vào và giờ ra thuộc cùng ngày lịch. Lượt qua ngày cần nhân viên nhập số tiền cùng lý do; chương trình lưu thông tin này để phân biệt với phí tự động. Các mức tiền là số nguyên VND, không có phần thập phân.

Các chức năng nhận dạng ảnh, điều khiển cổng, thanh toán điện tử, vé tháng, xác thực tài khoản và đồng bộ nhiều cổng nằm ngoài phạm vi. Những giới hạn đó giúp xác

định rõ điều mà kết quả kiểm thử có thể chứng minh: tính đúng của mô hình và quy trình trong bản console, không phải độ an toàn của một bãi xe đang vận hành thực tế.

1.4 Cách tiếp cận và cấu trúc báo cáo

Nhóm tổ chức phần phân tích theo một lượt gửi hoàn chỉnh: từ lúc nhận thẻ, ghi nhận xe vào, tra cứu xe ra cho đến khi lưu phí đã thu. Từ các thông tin cần nhớ và các điều kiện cần kiểm tra, chương trình được tách thành mô hình dữ liệu, dịch vụ nghiệp vụ, bộ tính phí, lưu trữ và giao diện. Các tình huống như trùng thẻ, bãi đầy hoặc ghi tệp thất bại được đưa vào cùng quá trình phân tích, vì chúng ảnh hưởng đến thời điểm dữ liệu được phép thay đổi.

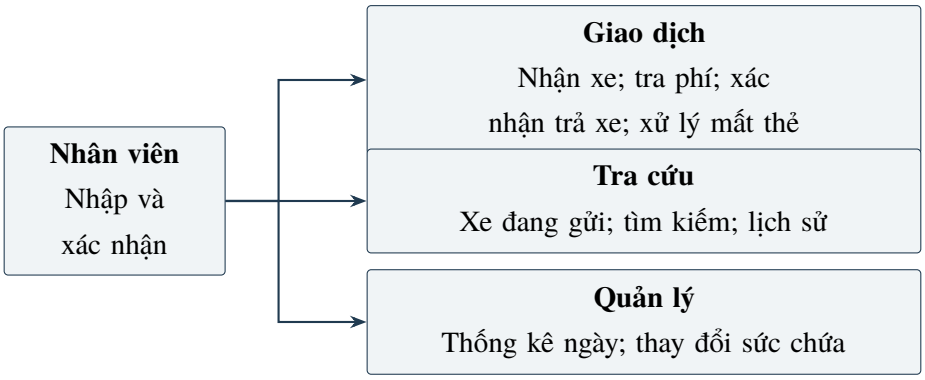
Các chương tiếp theo lần lượt mô tả yêu cầu, lựa chọn lớp, cách áp dụng OOP và quá trình cài đặt. Phần kiểm thử theo dõi một số ca quan trọng đến trạng thái cuối cùng để đối chiếu thiết kế với hành vi chạy thật. Ma trận đầy đủ của 45 ca kiểm thử được đặt ở phụ lục; phần chính dành chỗ cho việc giải thích kết quả và các giới hạn còn lại.

2. Yêu cầu phần mềm và nghiệp vụ

2.1 Người sử dụng và các nhóm chức năng

Người dùng trực tiếp là nhân viên trông xe. Nhân viên nhập mã thẻ, biển số, xác nhận việc thu tiền và xem thông tin quản lý. Sinh viên không có tài khoản trong hệ thống; phần mềm chỉ ghi nhận phương tiện, lượt gửi và thông tin sinh viên/chủ xe khi xử lý mất thẻ. Cách xác định này giúp tránh đưa thêm đăng nhập hoặc hồ sơ sinh viên thường xuyên; thông tin mất thẻ chỉ lưu trong lượt gửi tương ứng.

Hệ thống có ba nhóm công việc. Nhóm giao dịch gồm nhận và trả xe, làm thay đổi số xe trong bãi. Nhóm tra cứu gồm xem xe đang gửi, tìm kiếm và lịch sử, chỉ đọc dữ liệu. Nhóm quản lý gồm thống kê theo ngày và thay đổi sức chứa. Khi khởi động, chương trình đọc lại dữ liệu đã lưu; chỉ khi chưa có dữ liệu mới hỏi sức chứa ban đầu.



Hình 2.1. Các nhóm thao tác của nhân viên trong phạm vi hệ thống

Bảng 2.1. Danh sách yêu cầu chức năng

Mã	Chức năng	Kết quả cần đạt
F01	Khởi tạo	Tạo bãi có sức chứa hợp lệ khi chưa có dữ liệu.
F02	Nhận xe	Ghi thẻ, xe, giờ vào; kiểm tra trùng và chỗ trống.
F03	Trả xe	Tra lượt, đối chiếu, tính phí, xác nhận và lưu kết quả.
F04	Xe đang gửi	Hiển thị xe còn trong bãi, số xe và số chỗ trống.
F05	Tìm kiếm	Tìm các lượt theo mã thẻ hoặc biển số đã chuẩn hóa.
F06	Lịch sử	Xem cả lượt đang gửi và đã trả, phí và lý do thủ công.
F07	Thống kê	Đếm lượt vào/ra và doanh thu theo ngày.
F08	Sức chứa	Lưu sức chứa mới không nhỏ hơn số xe đang gửi.
F09	Khôi phục	Đọc lại lịch sử, lượt còn mở và mã lượt kế tiếp.
F10	Demo	Nhập thời gian để trình diễn, sử dụng dữ liệu riêng.
F11	Mất thẻ	Tra theo biển số, lưu thông tin sinh viên/chủ xe, thu phí thủ công.
F12	Danh sách mất thẻ	Xem các lượt đã xử lý mất thẻ.

2.2 Nhận xe: dữ liệu và điều kiện chấp nhận

Một thao tác nhận xe bắt đầu bằng mã thẻ, biển số và loại xe. Thời gian vào được lấy từ đồng hồ máy trong chế độ thường, hoặc do người trình diễn nhập trong chế độ demo. Nhân viên không cần cung cấp mã lượt; chương trình tự cấp mã mới khi tạo giao dịch.

Trước khi nhận, mã thẻ được bỏ khoảng trắng hai đầu và đổi sang chữ hoa. Biển số được đổi sang chữ hoa, bỏ khoảng trắng, dấu chấm và dấu gạch để so sánh. Nhờ vậy, nhập `29A1-123.45` hay `29a1 12345` không tạo thành hai phương tiện khác nhau. Bản mô phỏng kiểm tra ký tự và độ dài, chưa xác nhận biển số đó có được cấp ngoài thực tế hay không.

Thao tác chỉ được chấp nhận nếu thẻ chưa gắn với xe đang gửi, biển số chưa có trong bãi và số xe chưa đạt sức chứa. Các điều kiện này độc lập: còn chỗ không có nghĩa là được nhận bằng thẻ đang sử dụng. Ngược lại, một thẻ rảnh cũng không cho phép nhận thêm xe khi bãi đã đầy.

Bảng 2.2. Đặc tả thao tác nhận xe

Thành phần	Nội dung
Đầu vào	Mã thẻ, biển số, loại xe, thời gian vào.
Tiền điều kiện	Dữ liệu hợp lệ; thẻ và biển số chưa có lượt đang gửi; còn chỗ.
Luồng chính	Chuẩn hóa, kiểm tra, cấp mã lượt, tạo lượt và lưu trạng thái dự kiến.
Hậu điều kiện	Có thêm một lượt đang gửi; thẻ bận; số chỗ trống giảm một.
Luồng lỗi	Báo lỗi và quay lại menu; không thêm lượt, không tiêu hao mã lượt.
Hoàn tất	Chỉ thông báo giao thẻ cho sinh viên sau khi lưu thành công.

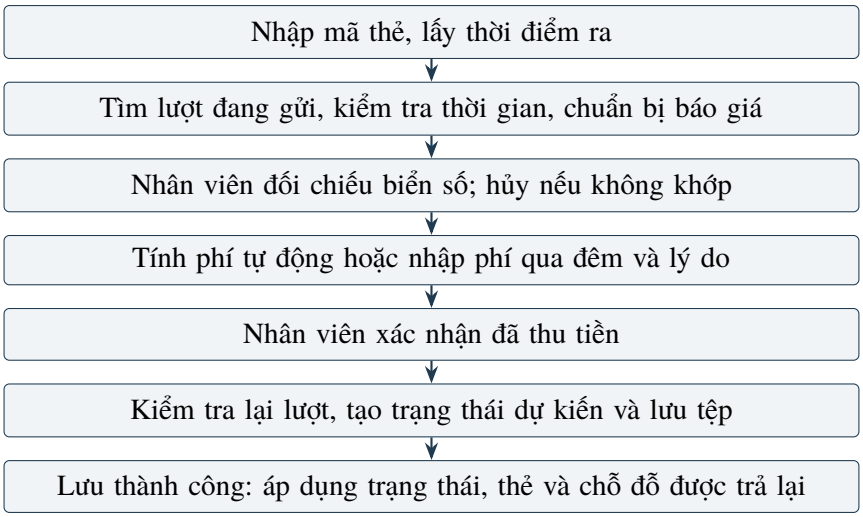
Ví dụ bãi có sức chứa 100 và đang giữ 99 xe. Nhận một xe bằng thẻ C001 làm số xe tăng lên 100 sau khi ghi tệp thành công. Nếu bước ghi không thành công, số xe vẫn là 99 và C001 chưa được xem là đã giao cho một lượt mới. Đặt điều kiện hoàn tất ở bước lưu giúp thông báo trên màn hình phù hợp với dữ liệu mà lần mở sau sẽ đọc được.

2.3 Trả xe và thời điểm ghi nhận thu tiền

Khi sinh viên trả thẻ, nhân viên nhập mã và hệ thống tìm lượt đang hoạt động của thẻ đó. Các lượt đã kết thúc có cùng mã thẻ vẫn được giữ trong lịch sử nhưng không được chọn để trả lần nữa. Thời điểm ra được lấy lúc tra thẻ, trước các bước đối chiếu và thu tiền.

Màn hình hiển thị biển số, loại xe, giờ vào, giờ ra và phí dự kiến. Nhân viên phải xác nhận biển số khớp xe thực tế. Nếu không khớp, thao tác dừng và dữ liệu giữ nguyên. Với lượt qua ngày, sau bước đối chiếu hệ thống yêu cầu số tiền và lý do. Cuối cùng, nhân viên xác nhận đã thu tiền để chương trình thực hiện đóng lượt.

Hai câu hỏi xác nhận có mục đích khác nhau. Xác nhận biển số kiểm tra sự phù hợp giữa xe và thẻ; xác nhận thu tiền xác định thời điểm cho phép kết thúc giao dịch. Câu trả lời ở bước đầu không được dùng thay cho bước sau. Một báo giá đã hiện trên màn hình cũng không phải bằng chứng rằng khoản tiền đã được thu.



Hình 2.2. Quy trình trả xe với hai bước xác nhận của nhân viên

Nếu nhân viên hủy sau khi xem phí, xe vẫn được tính vào số xe đang gửi. Khi quẹt lại thẻ, chương trình lấy thời gian mới và tính lại. Vì vậy, một lần xem lúc 18:59 có thể cho mức 3.000 đồng, nhưng lần quẹt tiếp theo lúc 19:00 sẽ chuyển sang 7.000 đồng nếu xe vào trước 19:00. Việc giữ nguyên báo giá cũ sau khi hủy sẽ làm sai quy tắc này.

Ở bước xác nhận, dịch vụ tìm lại lượt hiện tại và đối chiếu mã lượt, biển số, giờ vào với báo giá. Đây là cách ngăn một báo giá của lượt trước được áp dụng nhầm sau khi thẻ đã được tái sử dụng. Nếu kiểm tra hoặc lưu tệp thất bại, lượt hiện tại vẫn chưa đóng và chương trình chưa ghi nhận doanh thu.

2.4 Xử lý mất thẻ

Khi sinh viên làm mất thẻ, mã thẻ không còn là khóa tra cứu. Nhân viên nhập biển số để tìm lượt đang gửi, rồi ghi nhận giờ ra và thông tin trên giấy tờ. Mã SV/CCCD, họ tên sinh viên và họ tên chủ xe là bắt buộc; CCCD chủ xe, lớp, số điện thoại và ghi chú có thể bỏ trống. Vì sinh viên có thể đi xe của bố mẹ, thông tin sinh viên và chủ xe được lưu thành hai nhóm riêng.

Lượt mất thẻ luôn thu phí thủ công, kể cả khi vào và ra trong cùng ngày. Phí tự động, nếu có, chỉ hiển thị để nhân viên tham khảo. Lý do được ghi tự động là “Mất thẻ”. Thông tin được lưu cùng lượt gửi, không cần tệp ảnh riêng, và thẻ bị mất vẫn có thể dùng lại nếu tìm thấy.

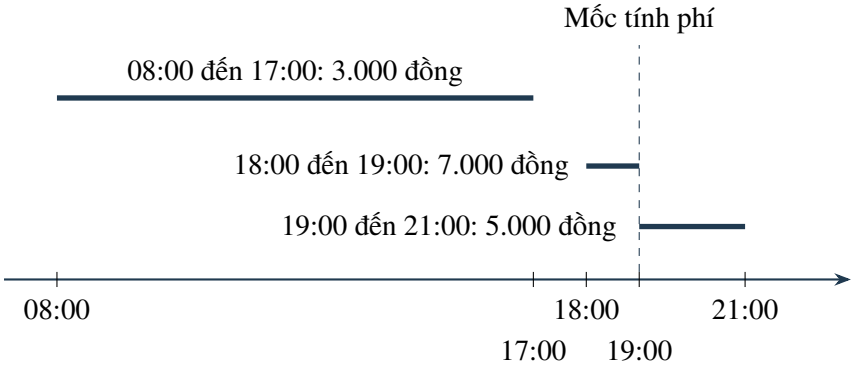
2.5 Biểu phí và các trường hợp biên

Biểu phí tự động được xét trên một lượt có thời gian vào và ra thuộc cùng ngày. Mốc phân chia là 19:00. Cụm từ ban ngày trong thiết kế chỉ nhóm quy tắc trước mốc này; chương trình chưa mô hình hóa lịch đóng, mở cổng theo từng ngày.

Bảng 2.3. Điều kiện lựa chọn mức phí cho xe máy và xe máy điện

Giờ vào	Giờ ra	Phí (VND)
Trước 19:00	Trước 19:00, cùng ngày	3.000
Từ 19:00	Không trước giờ vào, cùng ngày	5.000
Trước 19:00	Từ 19:00, cùng ngày	7.000
Bất kỳ giờ hợp lệ	Khác ngày vào	Nhập thủ công
Bất kỳ giờ hợp lệ	Bất kỳ giờ hợp lệ	Nhập thủ công do mất thẻ

Giá tính theo lượt, không nhân số giờ và không tách riêng sáng, chiều. Một xe vào 08:00 và ra 17:00 trả 3.000 đồng. Xe vào 18:00, ra 19:00 trả 7.000 đồng. Xe vào đúng 19:00, ra 21:00 trả 5.000 đồng. Dấu bằng ở mốc 19:00 cần được thể hiện trong cả mã nguồn và kiểm thử vì đây là nơi hai cách hiểu dễ khác nhau.



Hình 2.3. Ba lượt mẫu có cùng mốc phân chia 19:00

Ngày được lưu theo ngày lịch, bắt đầu lúc 00:00. Nếu xe vào ngày 26 và ra ngày 27, hệ thống chuyển sang thu phí thủ công dù hai thời điểm chỉ cách nhau ít phút. Chương trình không tự cộng thêm một vé ngày hoặc tự nhân phí với số ngày. Nhân viên nhập số tiền nguyên không âm và lý do để lịch sử ghi lại cách xử lý của lượt đó.

Giờ ra phải bằng hoặc sau giờ vào. Nếu bằng nhau, đây vẫn là một lượt gửi và áp dụng mức phí của khung tương ứng. Nếu giờ ra trước giờ vào, thao tác bị từ chối; chương

trình không tự đảo thời gian hoặc suy đoán rằng xe đã qua đêm. Quy tắc này đặc biệt cần trong chế độ demo, nơi người nhập có thể gõ nhầm ngày.

2.6 Tra cứu, thống kê và chất lượng dữ liệu

Xe đang gửi là lượt chưa có giờ ra. Chỗ trống bằng sức chứa trừ số lượt này. Sức chứa mới không được nhỏ hơn số xe hiện tại và chỉ được áp dụng sau khi lưu thành công.

Tra cứu so khớp chính xác sau chuẩn hóa và trả cả lịch sử. Một thẻ có thể có nhiều kết quả, nhưng tối đa một lượt đang gửi. Mã lượt giúp phân biệt những lần dùng lại thẻ.

Thống kê đếm lượt vào theo ngày vào, lượt ra và doanh thu theo ngày ra. Xe qua đêm có lượt vào ở ngày trước, còn lượt ra và phí ở ngày sau. Lượt mất thẻ đã trả cũng được tính doanh thu theo ngày ra. Lượt chưa trả hoặc báo giá bị hủy không tạo doanh thu.

Các quan hệ trên phải được kiểm tra lại khi đọc tệp. Phần thiết kế tiếp theo xác định lớp chịu trách nhiệm bảo vệ từng điều kiện, kể cả khi dữ liệu bị sửa ngoài chương trình.

3. Thiết kế lớp và đối tượng

3.1 Từ quy trình gửi xe đến mô hình lớp

Thông tin cần giữ sau một lần nhận xe gồm thẻ, phương tiện và thời điểm vào. Khi xe ra, chính lần gửi đó được bổ sung giờ ra và phí đã thu. Vì vậy, đối tượng trung tâm của mô hình là `ParkingSession`, tức một lượt gửi, thay vì chỉ là xe hoặc thẻ. Xe có thể quay lại nhiều lần; thẻ cũng được dùng lại; chỉ lượt gửi là đại diện cho một lần vào và ra cụ thể.

`Vehicle` lưu những gì thuộc về phương tiện trong phạm vi đề tài: biển số và loại xe. `ParkingLot` giữ sức chứa, danh sách các lượt và mã lượt kế tiếp. Các lớp này mô tả dữ liệu cùng quy tắc hợp lệ của dữ liệu, không đọc bàn phím hoặc in menu. Khi cần thực hiện một công việc có nhiều bước, `ParkingService` phối hợp mô hình với bộ tính phí và lớp lưu trữ.

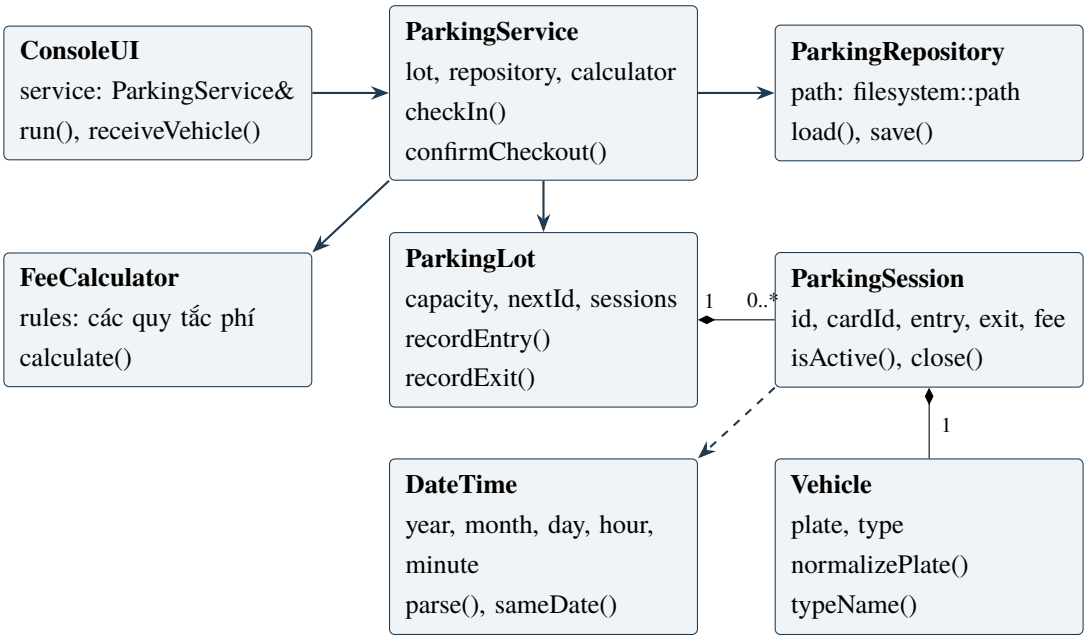
Việc tách lớp theo trách nhiệm làm rõ nơi phải sửa khi yêu cầu thay đổi. Đổi câu hỏi xác nhận trên màn hình thuộc về giao diện; đổi mức phí thuộc về các quy tắc phí; đổi định dạng tệp thuộc về lớp lưu trữ. Một thay đổi nghiệp vụ như điều kiện nhận xe có thể liên quan đến mô hình và dịch vụ, nhưng không cần đặt rải rác trong từng lựa chọn menu.

Bảng 3.1. Các lớp chính và phần thông tin do lớp quản lý

Lớp	Thông tin chính	Trách nhiệm
`Vehicle`	Biển số, loại xe	Chuẩn hóa và mô tả phương tiện.
`ParkingSession`	Thẻ, xe, giờ vào/ra, phí	Bảo vệ tính hợp lệ của một lượt.
`LostTicketInfo`	Mã SV/CCCD, họ tên SV, chủ xe, ghi chú	Kiểm tra thông tin khi mất thẻ.
`ParkingLot`	Sức chứa, lượt gửi, mã kế tiếp	Kiểm tra ràng buộc trên toàn bãi.
`DateTime`	Ngày, tháng, năm, giờ, phút	Kiểm tra và so sánh thời điểm.
`FeeCalculator`	Các đối tượng quy tắc phí	Tìm và gọi quy tắc phù hợp.
`ParkingService`	Bãi, bộ tính phí, lưu trữ	Điều phối giao dịch và truy vấn.
`ParkingRepository`	Đường dẫn tệp	Đọc, kiểm tra, ghi trạng thái bãi.
`ConsoleUI`	Tham chiếu đến dịch vụ	Nhận dữ liệu, xác nhận và hiển thị.

3.2 Quan hệ giữa các lớp

Một bãi chứa nhiều lượt gửi, mỗi lượt chứa một đối tượng xe. Những đối tượng này được lưu theo giá trị. Khi lượt được sao chép sang trạng thái dự kiến để chuẩn bị giao dịch, thông tin xe và thời gian của lượt cũng được sao chép theo. Không có một danh sách xe chung mà mọi lượt cùng sửa; biển số và loại xe của một lượt lịch sử vẫn giữ nguyên sau khi lượt đó kết thúc. `ParkingSession` có thể chứa một `LostTicketInfo` tùy chọn; thông tin này chỉ xuất hiện khi lượt được đóng do mất thẻ.



Hình 3.1. Sơ đồ các quan hệ chính; thuộc tính và phương thức được rút gọn

`ParkingService` sở hữu bãi xe, lớp lưu trữ và bộ tính phí. `ConsoleUI` chỉ giữ một tham chiếu đến dịch vụ đã được tạo trong `main()`. Giao diện có thể xem bãi qua tham chiếu hằng, nhưng không nhận một danh sách có thể sửa trực tiếp. Nhờ đó, các lựa chọn tìm kiếm hoặc hiển thị không vô tình thay đổi số xe và phí đã thu.

Một số kiểu dữ liệu được giữ ở mức đơn giản. `VehicleType` là `enum class` gồm xe máy và xe máy điện. Cả hai dùng cùng biểu phí và cùng quy trình nhận/trả, nên tách hai lớp con chỉ để đổi tên hiển thị chưa đem lại khác biệt hành vi cần thiết. Ngược lại, các trường hợp tính phí khác nhau thật sự về điều kiện áp dụng, nên được tổ chức thành một nhóm lớp kế thừa riêng.

3.3 Định danh: mã thẻ không phải mã lượt

Giả sử thẻ C001 được giao cho xe 29A1-12345 lúc 08:00 và nhận lại lúc 17:00. Đến 19:00, thẻ được giao cho một xe khác. Nếu dùng C001 làm khóa duy nhất của lịch sử, lần

ghi thứ hai sẽ phải thay thế bản ghi cũ hoặc khiến chương trình khó chọn bản ghi cần trả. Thẻ vẫn là cùng một vật, nhưng hai khoản phí thuộc hai lần gửi khác nhau.

Mô hình cấp một mã lượt tăng dần cho mỗi lần nhận. Mã thẻ là một thuộc tính của lượt, có thể lặp trong lịch sử nhưng không lặp giữa các lượt đang hoạt động. Khi tra trả xe, chương trình lọc lượt chưa có giờ ra và có mã thẻ phù hợp. Khi xem lịch sử, tất cả lượt của thẻ được trả về, phân biệt bằng mã lượt và thời gian.

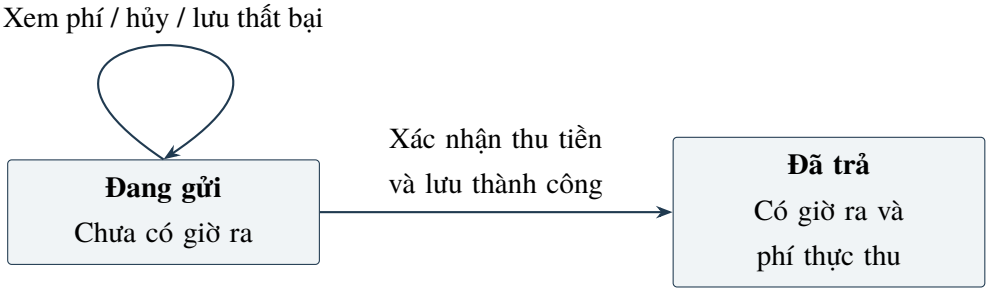
Bảng 3.2. Ví dụ thẻ được dùng lại trong các lượt độc lập

Lượt	Thẻ	Biển số	Vào / ra ngày 26/10	Phí
1	C001	29A112345	08:00 / 17:00	3.000
2	C001	29M99999	19:00 / 21:00	5.000
3	C002	29B188888	18:00 / chưa ra	0

Số 0 ở lượt đang gửi chỉ biểu thị chưa thu tiền, không có nghĩa là lượt được miễn phí. Việc phân biệt này được hỗ trợ bởi giờ ra tùy chọn và cờ phí thủ công, thay vì suy luận trạng thái chỉ từ số tiền. Một lượt qua đêm đã trả có thể có phí 0 nếu có lý do hợp lệ, còn một lượt đang gửi luôn chưa hoàn tất thu tiền.

3.4 Trạng thái của lượt và đối tượng báo giá

`ParkingSession` chỉ có hai trạng thái lưu trữ: đang gửi và đã trả. Trạng thái được suy ra từ việc có hay chưa có giờ ra. Cách biểu diễn này tránh lưu song song một cờ `paid` hoặc `closed` có thể không khớp với thời gian. Khi đóng lượt, giờ ra, phí và thông tin thu thủ công được kiểm tra rồi cập nhật cùng nhau. Lượt mất thẻ vẫn là trạng thái đã trả; nó chỉ mang thêm thông tin sinh viên/chủ xe và cờ phí thủ công.



Hình 3.2. Trạng thái lưu trữ của một lượt gửi

Xem báo giá không tạo trạng thái thứ ba trong tệp. Dịch vụ trả về `CheckoutQuote`, một đối tượng tạm để giao diện hiển thị. Đối tượng này có mã lượt, mã thẻ, biển số, giờ

vào/ra, phí tùy chọn và dấu hiệu cần thu thủ công. Với lượt qua ngày, phí chưa có giá trị cho đến khi nhân viên nhập; với lượt cùng ngày, phí được tính ngay để xem trước.

Khi xác nhận, dịch vụ không dùng riêng mã thẻ và số tiền từ báo giá. Nó tìm lại lượt hiện tại, kiểm tra định danh rồi tính lại phí. Điều này giữ sự phân biệt giữa dữ liệu dùng để trình bày và dữ liệu có quyền quyết định giao dịch. Trong bản một phiên, hai bước thường diễn ra liên tiếp, nhưng phép kiểm tra vẫn cần để tránh dùng lại một đối tượng báo giá đã cũ trong lời gọi nghiệp vụ.

3.5 Ràng buộc và giao diện nghiệp vụ

Ràng buộc được đặt ở nơi có đủ thông tin để kiểm tra. `Vehicle` biết cách chuẩn hóa biển số, nhưng không thể biết xe đã có trong bãi hay chưa. `ParkingSession` kiểm tra giờ ra và thông tin thu tiền, còn `ParkingLot` kiểm tra thẻ, biển số đang dùng và sức chứa. `ParkingLot::findActiveByPlate()` bổ sung đường tra cứu khi không có mã thẻ. Dịch vụ kiểm tra sự phù hợp của báo giá với lượt hiện tại và điều phối việc lưu.

Bảng 3.3. Giao diện nghiệp vụ được giao diện console sử dụng

Thao tác	Ý nghĩa
`checkIn()`	Nhận dữ liệu, tạo lượt và chỉ trả mã lượt sau khi lưu được.
`prepareCheckout()`	Kiểm tra thẻ, thời gian, tạo báo giá; không ghi tệp.
`confirmCheckout()`	Kiểm tra lại lượt, xác định phí, lưu rồi áp dụng kết quả.
`prepareLostTicketCheckout()`	Tra theo biển số, tạo báo giá mất thẻ; không ghi tệp.
`confirmLostTicketCheckout()`	Kiểm tra lại lượt, lưu thông tin mất thẻ và phí thủ công.
`getLostTicketSessions()`	Lọc các lượt đã xử lý mất thẻ.
`searchByCard()`	Tìm lịch sử chính xác theo mã thẻ chuẩn hóa.
`searchByPlate()`	Tìm lịch sử chính xác theo biển số chuẩn hóa.
`statistics()`	Đếm lượt và cộng doanh thu của ngày hợp lệ.
`changeCapacity()`	Kiểm tra và lưu sức chứa mới.

Các thuộc tính quan trọng nằm trong vùng `private`; giao diện không có setter để tự gán số xe, phí hoặc trạng thái trả. Các phương thức đọc dùng `const` khi phù hợp. Điều này không chỉ là quy ước đặt từ khóa: nó giữ các thao tác chỉ đọc tách khỏi các thao tác phải kiểm tra và lưu dữ liệu.

`ParkingLot` dùng một `vector` chứa cả lịch sử và các lượt đang gửi. Việc tra cứu hiện tại duyệt danh sách và lọc theo điều kiện; các tập mã dùng để phát hiện trùng khi

khôi phục chỉ được tạo trong bước kiểm tra. Chương trình chưa duy trì thêm bảng chỉ mục phải đồng bộ sau mỗi giao dịch. Đổi lại, thời gian tìm kiếm và ghi dữ liệu tăng theo lịch sử, là giới hạn được đánh giá ở Chương 6.

4. Vận dụng lập trình hướng đối tượng

4.1 Đóng gói: giữ đối tượng ở trạng thái hợp lệ

Một biển số được nhập từ bàn phím có thể có dấu chấm, dấu gạch hoặc chữ thường. Nếu mỗi nơi tự chuẩn hóa theo một cách, việc so trùng sẽ không còn đáng tin. Vì vậy, `Vehicle` thực hiện chuẩn hóa trong hàm tạo. Khi một đối tượng xe đã tồn tại, các lớp còn lại có thể dùng biển số của nó để so sánh mà không phải lặp lại toàn bộ kiểm tra ký tự.

Đoạn mã 4.1. Giao diện lớp Vehicle

```
class Vehicle {
private:
    std::string plate_;
    VehicleType type_;

public:
    Vehicle(const std::string& plate, VehicleType type);
    static std::string normalizePlate(const std::string& plate);
    const std::string& getPlate() const;
    VehicleType getType() const;
    std::string typeName() const;
};
```

Hai thuộc tính nằm ở vùng riêng. Giao diện được phép lấy biển số và loại xe, không được gán trực tiếp một chuỗi mới vào `plate_`. Các phương thức đọc là `const`; điều này thể hiện rằng việc hiển thị loại xe hoặc lấy biển số không phải một thao tác chỉnh sửa dữ liệu.

Đóng gói cũng áp dụng cho các thay đổi có nhiều thuộc tính liên quan. Khi trả xe, không thể chỉ đặt giờ ra rồi để phí được cập nhật ở một lựa chọn menu khác. `ParkingSession::close()` nhận các thông tin cần thiết, kiểm tra điều kiện rồi mới lưu chúng. Một lượt đã kết thúc không được đóng lần nữa; phí âm hoặc giờ ra trước giờ vào đều bị từ chối.

Đoạn mã 4.2. Kiểm tra và kết thúc một lượt gửi

```
void ParkingSession::close(const DateTime& exit, long long fee, bool manual,
                           const std::string& reason) {
    if (!isActive()) throw ConflictException("Lượt gửi đã kết thúc.");
    if (exit < entry_) throw ValidationException("Giờ ra không được trước giờ vào.");
    if (fee < 0) throw ValidationException("Phí không được âm.");
    const bool overnight = !entry_.sameDate(exit);
    if (manual != overnight) {
        throw ValidationException("Chỉ lượt qua ngày mới được nhập phí thủ công.");
    }
    const std::string normalizedReason = trim(reason);
    if ((manual && normalizedReason.empty()) || (!manual && !normalizedReason.empty()) ||
        normalizedReason.size() > 500 ||
        normalizedReason.find_first_of("\r\n") != std::string::npos) {
        throw ValidationException("Lý do phí thủ công cần từ 1 đến 500 ký tự trên một dòng.");
    }
    exit_ = exit;
    paidFee_ = fee;
    manualFee_ = manual;
    reason_ = normalizedReason;
}
```

Đáng chú ý là cờ thu phí thủ công phải phù hợp với việc có qua ngày hay không. Chỉ gán một số tiền dương chưa đủ để hợp lệ: với lượt trả bình thường, một lượt cùng ngày có cờ thủ công là sai quy tắc, còn lượt qua ngày thiếu lý do cũng chưa được chấp nhận. Những kiểm tra này nằm cùng thao tác đóng lượt để không phụ thuộc nhân viên đã đi qua menu nào.

Với lượt mất thẻ, `ParkingSession::closeLostTicket()` là thao tác đóng riêng. Phương thức vẫn kiểm tra giờ ra, phí không âm và lý do, nhưng cho phép phí thủ công trong cùng ngày và gắn `LostTicketInfo`. Nhờ vậy, quy tắc của lượt bình thường không bị nới lỏng.

Các setter rời rạc như `setExit()`, `setFee()` hay `setClosed()` sẽ khiến phía gọi phải tự nhớ thứ tự và điều kiện. Thiết kế hiện tại dùng một thao tác mang ý nghĩa nghiệp vụ để gom trách nhiệm đó về lớp lượt gửi. Ở cấp toàn bãi, `recordEntry()` và `changeCapacity()` cũng thực hiện cùng cách tiếp cận với thẻ, biển số và sức chứa.

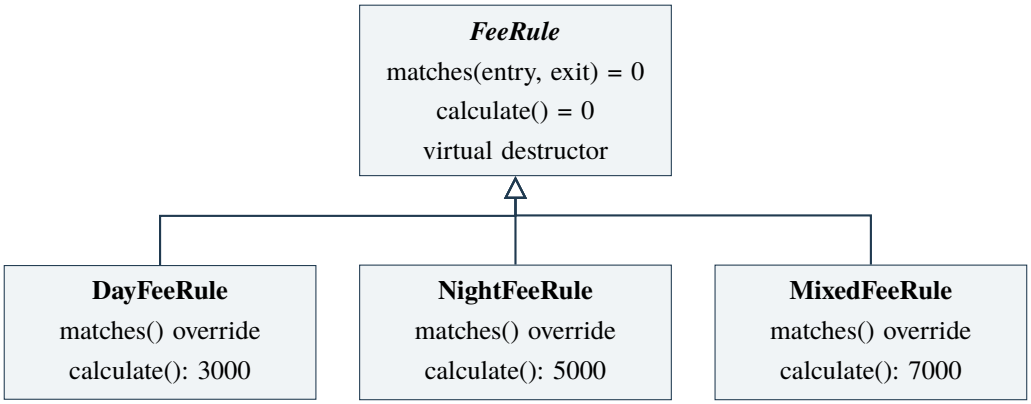
4.2 Kế thừa tại các quy tắc tính phí

Ba trường hợp phí đều cần trả lời hai câu hỏi: quy tắc có áp dụng cho cặp thời gian này không, và nếu có thì thu bao nhiêu. Phần giống nhau đó trở thành giao diện `FeeRule`. Mỗi lớp con tự xác định điều kiện phù hợp và mức tiền của mình.

Đoạn mã 4.3. Lớp cha trừu tượng của các quy tắc phí

```
class FeeRule {
public:
    virtual bool matches(const DateTime& entry, const DateTime& exit) const
    ↪  = 0;
    virtual int calculate() const = 0;
    virtual ~FeeRule() = default;
};
```

Hai phương thức có `= 0` là thuần ảo, nên chương trình không tạo một quy tắc chung chưa có cách tính cụ thể. Các lớp `DayFeeRule`, `NightFeeRule` và `MixedFeeRule` cài đặt đầy đủ giao diện và dùng `override`. Nếu tên hoặc danh sách tham số không khớp, trình biên dịch có thể báo ngay thay vì âm thầm tạo một hàm khác.



Hình 4.1. Các lớp con cùng cung cấp giao diện kiểm tra và tính phí

Quy tắc ban ngày khớp khi lượt cùng ngày và giờ ra trước 19:00. Quy tắc buổi tối khớp khi giờ vào từ 19:00; vì thời gian ra không trước giờ vào nên xe cũng ra trong khung tối. Quy tắc kết hợp khớp khi vào trước 19:00 nhưng ra từ mốc đó. Mỗi lớp còn kiểm tra cùng ngày và thứ tự thời gian để điều kiện tự nó đã có nghĩa rõ ràng.

Kế thừa ở đây diễn tả một quan hệ thay thế: một quy tắc ban ngày có thể được dùng ở nơi cần một `FeeRule`. Quan hệ này khác với việc bãi xe chứa các lượt gửi. Lượt gửi không phải một loại bãi xe, nên không kế thừa từ bãi. Việc chọn đúng loại quan hệ giúp sơ đồ lớp phản ánh công việc thực tế thay vì tạo một cây kế thừa chỉ để có nhiều tầng.

Các mức phí hiện là hằng số, nên có thể cài đặt toàn bộ bằng một hàm điều kiện ngắn. Nhóm vẫn tách quy tắc để minh họa một điểm mở rộng có nội dung thực: khi cách chọn mức tiền thay đổi, lớp quy tắc chịu trách nhiệm cho điều kiện đó. Đổi lại, cách tổ chức này có nhiều lớp hơn. Với quy mô bài tập, sự đánh đổi được chấp nhận để làm rõ cơ chế kế thừa và đa hình, không phải để khẳng định mọi bảng giá đều cần mô hình tương tự.

4.3 Đa hình trong một lần tính tiền

`FeeCalculator` giữ một danh sách `unique_ptr<FeeRule>`. Kiểu được nhìn thấy qua phần tử danh sách là giao diện lớp cha, còn đối tượng được tạo thực tế thuộc một trong ba lớp con. Vòng lặp không ép kiểu để gọi riêng từng lớp; nó gọi `matches()` và `calculate()` qua cùng giao diện.

Đoạn mã 4.4. Khởi tạo đối tượng lớp con và gọi qua lớp cha

```
#include "pricing/FeeCalculator.h"
#include "exceptions/ParkingException.h"

FeeCalculator::FeeCalculator() {
    rules_.push_back(std::make_unique<DayFeeRule>());
    rules_.push_back(std::make_unique<NightFeeRule>());
    rules_.push_back(std::make_unique<MixedFeeRule>());
}

int FeeCalculator::calculate(const DateTime& entry, const DateTime& exit)
    const {
    if (exit < entry) throw ValidationException("Giờ ra không được trước
        giờ vào.");
    if (!entry.sameDate(exit)) throw ValidationException("Lượt qua ngày cần
        nhập phí và lý do.");
    for (const auto& rule : rules_) {
        // Lời gọi qua lớp cha được phân giải tới đối tượng lớp con tại
            thời điểm chạy.
        if (rule->matches(entry, exit)) return rule->calculate();
    }
    throw ValidationException("Không tìm thấy quy tắc tính phí phù hợp.");
}
```

Xét lượt vào lúc 18:00 và ra lúc 19:00 ngày 26/10. Bộ tính phí kiểm tra thứ tự thời gian và cùng ngày trước khi duyệt quy tắc. Đối tượng ban ngày trả về không khớp vì giờ ra không còn trước 19:00. Đối tượng buổi tối không khớp vì giờ vào chưa đến 19:00. Đối tượng kết hợp khớp và phương thức `calculate()` của chính lớp đó trả về 7.000.

Bảng 4.1. Theo dõi các lời gọi cho lượt 18:00 đến 19:00

Đối tượng thực	Kết quả kiểm tra	Hành động tiếp theo
`DayFeeRule`	Giờ ra không trước 19:00	Bỏ qua
`NightFeeRule`	Giờ vào trước 19:00	Bỏ qua
`MixedFeeRule`	Vào trước, ra từ 19:00	Trả 7.000

Nếu đổi giờ vào thành 19:00 và giờ ra thành 21:00, vẫn vòng lặp đó nhưng đối tượng buổi tối được chọn và trả về 5.000. Sự khác biệt nằm ở đối tượng lớp con xử lý lời gọi ảo. Đây là đa hình tại thời điểm chạy, không phải nạp chồng hàm dựa trên danh sách tham số.

Các điều kiện phải không chồng lấn. Nếu hai lớp cùng khớp, kết quả sẽ phụ thuộc thứ tự trong danh sách. Nếu không lớp nào khớp một lượt cùng ngày hợp lệ, chương trình sẽ không tính được tiền. Vì vậy, kiểm thử không chỉ lấy ba ví dụ điển hình mà còn kiểm tra mọi cặp phút hợp lệ trong ngày. Vai trò của phép kiểm tra đó được trình bày cụ thể ở Chương 6.

Lượt qua ngày được kiểm tra trước và không đi vào việc chọn ba mức tự động. Dịch vụ yêu cầu phí thủ công cùng lý do thay vì để một quy tắc cùng ngày vô tình trả về mức tiền. Phân biệt ở ranh giới này giữ cho bộ tính phí chỉ trả lời những trường hợp đã có chính sách rõ ràng.

4.4 Ngoại lệ và đường đi của lỗi

Nhận xe có thể thất bại vì nhiều nguyên nhân. Thẻ đang dùng là xung đột dữ liệu; bãi đầy là giới hạn sức chứa; không mở được tệp là lỗi lưu trữ. Nếu tất cả chỉ trả về `false`, giao diện khó giải thích nên nhập lại thông tin hay kiểm tra việc lưu. Chương trình dùng các lớp ngoại lệ để mang loại lỗi và thông điệp đến nơi xử lý.

Bảng 4.2. Phân loại lỗi trong mô hình và lưu trữ

Ngoại lệ	Ví dụ phát sinh	Trạng thái cần giữ
`ValidationException`	Ngày sai, phí âm, thiếu lý do	Không áp dụng dữ liệu sai.
`NotFoundException`	Thẻ không có lượt đang gửi	Không đóng lượt khác.
`ConflictException`	Trùng thẻ/xe hoặc báo giá cũ	Không tạo hoặc kết thúc nhầm lượt.
`CapacityException`	Bãi đầy, giảm sức chứa quá mức	Giữ số xe và sức chứa cũ.
`StorageException`	Không ghi được tệp hoặc tệp hỏng	Không báo giao dịch thành công.

Các lớp này kế thừa ParkingException; lớp gốc kế thừa std::runtime_error. Đây là nhóm kế thừa thứ hai của chương trình. Giao diện có thể bắt chung theo tham chiếu lớp gốc mà vẫn nhận thông điệp của lỗi cụ thể qua what(). Bắt theo tham chiếu cũng tránh sao chép ngoại lệ thành một đối tượng chỉ còn phần lớp cha. Riêng LostTicketInfo kiểm tra mã SV/CCCD, họ tên sinh viên và họ tên chủ xe ngay trong hàm tạo, nên lỗi thiếu thông tin được phát hiện trước khi lượt bị đóng.

Đoạn mã 4.5. Lớp ngoại lệ gốc và các nhóm lỗi

```
class ParkingException : public std::runtime_error {
public:
    explicit ParkingException(const std::string& message) :
    ↪ std::runtime_error(message) {}
};

class ValidationException : public ParkingException { using
↪ ParkingException::ParkingException; };
class NotFoundException : public ParkingException { using
↪ ParkingException::ParkingException; };
class ConflictException : public ParkingException { using
↪ ParkingException::ParkingException; };
class CapacityException : public ParkingException { using
↪ ParkingException::ParkingException; };
class StorageException : public ParkingException { using
↪ ParkingException::ParkingException; };
```

Ví dụ, nhân viên nhận xe thứ hai bằng C001 trong khi thẻ đang gắn với một xe. ParkingLot::recordEntry() ném ConflictException trước khi thêm phần tử. Ngoại lệ truyền ra khỏi checkIn() đến vòng lặp giao diện. Nhánh bắt lỗi in thông điệp, sau đó menu xuất hiện lại. Không có lượt mới và mã lượt kế tiếp vẫn giữ nguyên.

Một lỗi khác xảy ra sau khi đã tạo được trạng thái dự kiến: không ghi được tệp tạm. Lúc này mô hình dự kiến có xe mới, nhưng dịch vụ chưa thay thế bãi đang chạy. Ngoại lệ lưu trữ làm hàm giao dịch kết thúc trước bước áp dụng. Nhân viên nhận được lỗi; lần xem bãi tiếp theo vẫn thấy dữ liệu cũ. Hai lỗi phát sinh ở hai tầng khác nhau nhưng đều không được để lại một giao dịch thành công một phần.

Hủy trả xe không được xem là ngoại lệ. Nhân viên có thể chủ động không xác nhận biển số hoặc thu tiền, nên giao diện dùng nhánh điều kiện để dừng thao tác. Ngoại lệ được dành cho dữ liệu và điều kiện không thỏa mãn, giúp ý nghĩa của các đường xử lý rõ hơn.

4.5 Quản lý tài nguyên và tính nhất quán

Các đối tượng quy tắc được quản lý bằng `unique_ptr`, mỗi con trỏ sở hữu duy nhất một đối tượng. Khi bộ tính phí bị hủy, tài nguyên của các quy tắc được giải phóng tự động. Hàm hủy của `FeeRule` là ảo để việc hủy qua kiểu lớp cha thực hiện đúng đối với lớp con. Chương trình không cần một vòng lặp `delete` tách rời ở cuối luồng chạy.

Luồng tệp cũng được giữ trong các đối tượng có phạm vi sống rõ ràng. Tuy vậy, việc tự đóng tài nguyên khi ra khỏi phạm vi không thay cho kiểm tra ghi thành công. Lớp lưu trữ vẫn phải kiểm tra sau khi mở, ghi, đẩy dữ liệu và đóng tệp. Chỉ dựa vào việc không có ngoại lệ tự phát sẽ bỏ sót trường hợp luồng ghi đã chuyển sang trạng thái lỗi.

Đoạn mã 4.6. Lưu trạng thái dự kiến trước khi thay thế bãi đang chạy

```
void ParkingService::commit(ParkingLot candidate) {
    // Chỉ thay đổi dữ liệu đang chạy sau khi lưu thành công.
    repository_.save(candidate);
    lot_ = std::move(candidate);
}
```

Hàm `commit()` ngắn nhưng xác định thứ tự của giao dịch. Dữ liệu ứng dụng chỉ được thay sau khi `save()` trả về thành công. Nhận xe, trả xe và đổi sức chứa đều đi qua điểm này, nên không có ba cách lưu khác nhau trong ba lựa chọn menu.

Đây là cách giữ nhất quán ở quy mô một phiên với dữ liệu nhỏ. Mỗi giao dịch phải sao chép bãi và ghi toàn bộ lịch sử, nên không phù hợp khi lịch sử tăng rất lớn. Báo cáo giữ cả lợi ích lẫn chi phí của lựa chọn này: dễ đọc, dễ thử lỗi và tránh cập nhật nửa chừng trong các tình huống đã mô phỏng, nhưng chưa có giao dịch đồng thời hay bảo đảm phục hồi như một cơ sở dữ liệu.

5. Cài đặt và xử lý dữ liệu

5.1 Tổ chức chương trình và giao diện

Mã C++ nằm trong `source_code/`. Thư mục khai báo và cài đặt được chia cùng nhóm: mô hình, tính phí, dịch vụ, lưu trữ, ngoại lệ và giao diện. Tệp `main.cpp` tạo lớp lưu trữ, dịch vụ và giao diện, sau đó chạy vòng lặp menu. Các lớp nghiệp vụ không cần biết đang được gọi từ menu hay từ chương trình kiểm thử.

Bảng 5.1. Các điểm vào của giao diện console

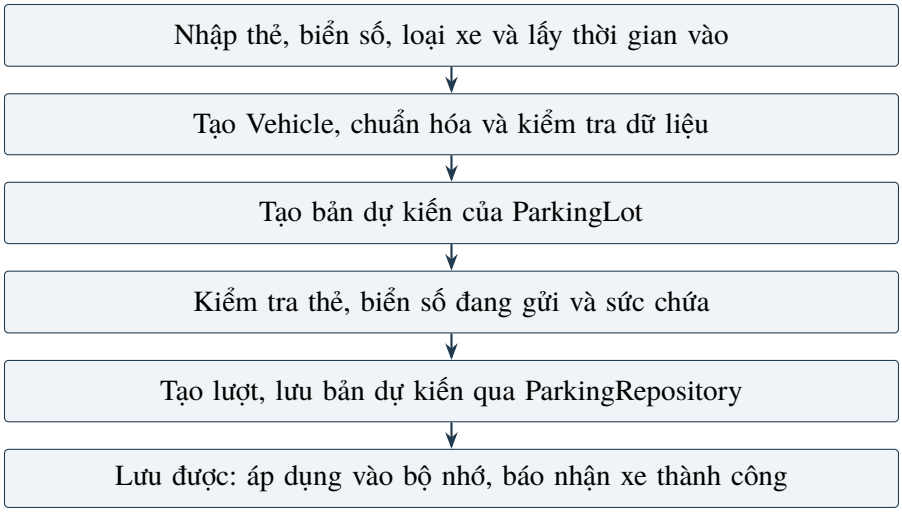
Lựa chọn	Công việc	Tác động dữ liệu
1	Nhận xe	Tạo lượt đang gửi nếu lưu thành công.
2	Trả xe	Có thể chỉ xem; chỉ đóng lượt sau xác nhận.
3	Xe đang gửi	Chỉ đọc số xe, sức chứa và các lượt chưa ra.
4	Tìm kiếm	Chỉ đọc theo thẻ hoặc biển số.
5	Lịch sử	Chỉ đọc toàn bộ lượt.
6	Thống kê	Chỉ đọc và tính theo ngày yêu cầu.
7	Đổi sức chứa	Kiểm tra và lưu cấu hình mới.
8	Xử lý mất thẻ	Tra theo biển số; lưu thông tin sinh viên/chủ xe; thu phí thủ công.
9	Danh sách mất thẻ	Chỉ đọc các lượt đã xử lý mất thẻ.
0	Thoát	Không tạo thêm giao dịch.

Giao diện đọc theo dòng, sau đó kiểm tra số nguyên và miền giá trị. Cách này tránh việc một ký tự sai làm luồng nhập số giữ trạng thái lỗi qua nhiều lần lặp. Khi dữ liệu không hợp lệ, thao tác hiện tại dừng và menu hiện lại. Khi luồng nhập kết thúc, chương trình thoát thay vì tiếp tục hỏi trên một luồng không còn dữ liệu.

Các câu xác nhận yêu cầu nhập `y` hoặc `Y`. Câu trả lời khác được hiểu là chưa đồng ý. Quy tắc này khiến việc nhấn Enter không tự đóng một lượt gửi. Trên Windows, chương trình thiết lập bảng mã console UTF-8 để thông báo tiếng Việt; phần thiết lập hệ điều hành được giữ trong tệp khởi động, không đưa vào các lớp bãi xe hoặc tính phí.

5.2 Luồng xử lý nhận xe

Sau khi nhận thẻ, biển số và loại xe, giao diện tạo `Vehicle`. Nếu biển số không hợp lệ, lỗi được phát hiện trước khi gọi dịch vụ nhận xe. Thời điểm vào được lấy theo chế độ chạy. Dịch vụ sao chép `ParkingLot` rồi gọi `recordEntry()` trên bản sao để kiểm tra toàn bãi và thêm lượt.



Hình 5.1. Luồng nhận xe; chỉ bước cuối thay trạng thái đang chạy

Trong bản dự kiến, chương trình kiểm tra thẻ và biển số qua các lượt đang hoạt động, sau đó kiểm tra số xe so với sức chứa. Mã lượt chỉ tăng sau khi thêm được đối tượng mới vào danh sách. Dịch vụ chuyển bản dự kiến đến điểm lưu chung; thông báo nhận xe thành công xuất hiện sau khi việc lưu hoàn tất.

Thứ tự này có một chi tiết dễ bỏ sót: bản sao có thể đã chứa lượt mới và mã kế tiếp mới, nhưng bãi đang chạy vẫn chưa thay đổi. Vì vậy, khi không tạo được tệp tạm, cả mã lượt lẫn số xe của bãi thật vẫn giữ nguyên. Khi nhân viên làm lại thao tác sau khi xử lý lỗi, chương trình không để lại một lượt rỗng hoặc tiêu hao mã vì lần nhận trước chưa hoàn tất.

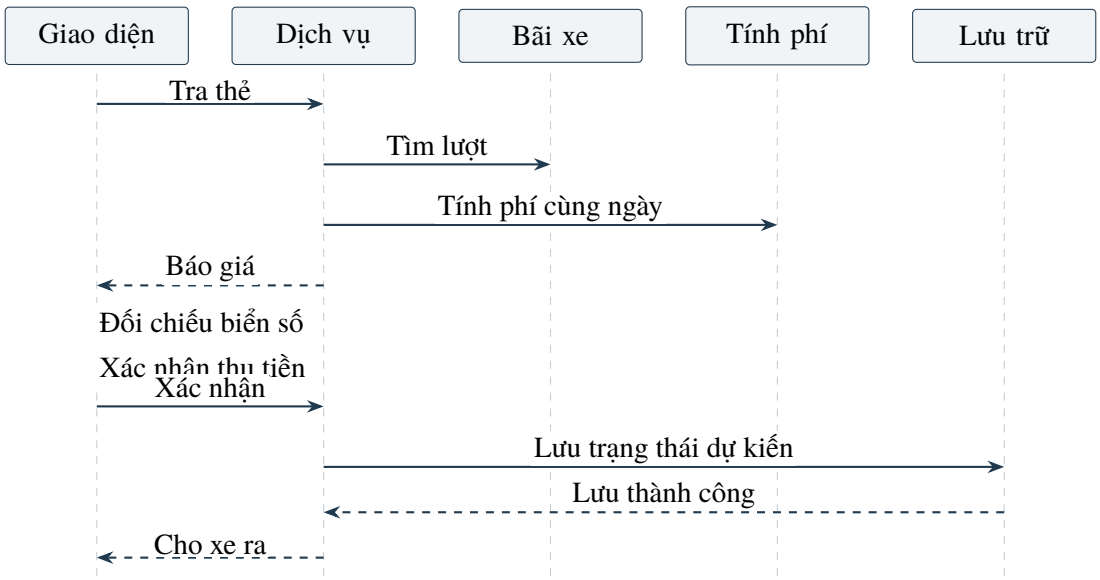
5.3 Từ tra thẻ đến xác nhận trả xe

Phương thức chuẩn bị trả xe lấy lượt đang gửi, kiểm tra thứ tự thời gian và xác định có qua ngày hay không. Nếu cùng ngày, bộ tính phí trả về số tiền. Nếu khác ngày, báo giá đánh dấu cần nhập thủ công và để trống giá trị phí tự động.

Đoạn mã 5.1. Chuẩn bị thông tin trả xe nhưng chưa sửa dữ liệu

```
CheckoutQuote ParkingService::prepareCheckout(const std::string& cardId,
    const DateTime& exit) const {
    const auto& session = lot_.findActiveByCard(cardId);
    if (exit < session.getEntry()) throw ValidationException("Giờ ra không
        được trước giờ vào.");
    const bool overnight = !session.getEntry().sameDate(exit);
    std::optional<long long> fee;
    if (!overnight) fee = calculator_.calculate(session.getEntry(), exit);
    return {session.getId(), session.getCardId(),
        session.getVehicle().getPlate(),
            session.getEntry(), exit, fee, overnight};
}
```

Phương thức được khai báo `const` vì không đóng lượt hoặc sửa bãi. Kết quả chỉ là dữ liệu để giao diện hỏi nhân viên. Với một lượt cùng ngày, trình tự gọi được mô tả ở Hình 5.2. Hai khoảng làm việc của nhân viên nằm giữa bước xem báo giá và bước xác nhận.



Hình 5.2. Trình tự rút gọn của một lượt trả xe cùng ngày

Khi gọi `confirmCheckout()`, dịch vụ tra lại theo thẻ và thời gian ra, rồi so sánh mã lượt, biển số và giờ vào. Phí cùng ngày được lấy từ kết quả tính lại; số tiền do đối tượng báo giá mang đến không được tin ngay. Với lượt qua ngày, số tiền và lý do được kiểm tra trước khi ghi vào lượt dự kiến.

Nhân viên xác nhận thu tiền trước khi dịch vụ lưu dữ liệu, nhưng thông báo cho xe ra chỉ xuất hiện sau khi lưu thành công. Nếu bước lưu thất bại, màn hình báo lỗi và xe vẫn ở trạng thái đang gửi. Khi đó, khoản tiền nhân viên đã nhận cần được ghi nhận riêng.

Khi mất thẻ, giao diện chuyển sang tra theo biển số. Dịch vụ tạo báo giá với cờ cần thu thủ công. Giao diện nhập thông tin sinh viên/chủ xe và phí, sau đó dịch vụ kiểm tra lại lượt, lưu và đóng. Thông tin sinh viên và chủ xe nằm trong `LostTicketInfo`.

5.4 Tệp dữ liệu và cách khôi phục

Lớp lưu trữ dùng tệp văn bản để nhóm dễ quan sát dữ liệu khi kiểm tra. Dòng đầu ghi sức chứa và mã lượt kế tiếp; các dòng sau chứa từng lượt gửi. Cấu trúc này đủ cho chương trình chạy trên một máy và không cần cài thêm hệ quản trị cơ sở dữ liệu.

Bảng 5.2. Các trường của một lượt trong tệp dữ liệu

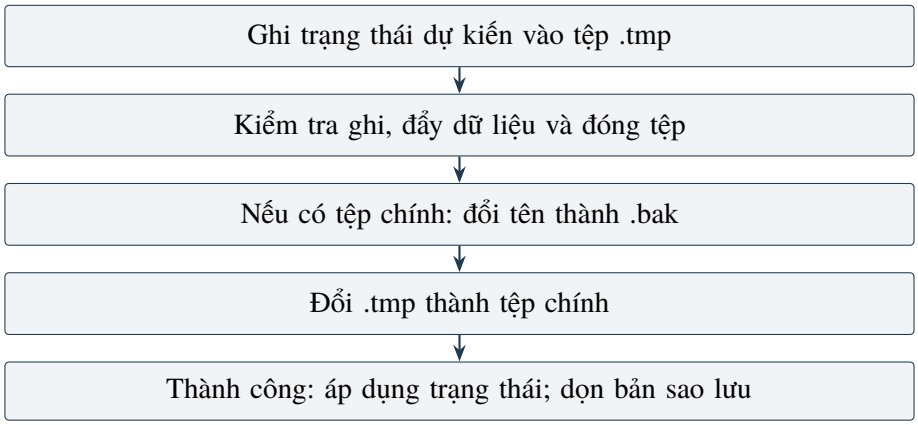
Trường	Cách hiểu khi đọc lại
Mã lượt	Số nguyên dương, không trùng và nhỏ hơn mã kế tiếp.
Mã thẻ, biển số	Chuỗi được đọc rồi chuẩn hóa, kiểm tra hợp lệ.
Loại xe	1 cho xe máy, 2 cho xe máy điện.
Thời gian vào	Chuỗi ngày giờ có độ chính xác đến phút.
Thời gian ra	Ngày giờ hợp lệ, hoặc - nếu đang gửi.
Phí đã thu	Số nguyên không âm; lượt đang gửi có giá trị 0.
Cờ thủ công	0 hoặc 1; lượt thường chỉ bật khi qua ngày, lượt mất thẻ luôn bật.
Lý do	Chuỗi một dòng; bắt buộc khi thu thủ công.
Thông tin mất thẻ	Cờ 0/1; nếu 1: mã SV/CCCD, họ tên SV, họ tên chủ xe, CCCD chủ xe, lớp, SĐT, ghi chú.

Các chuỗi được ghi bằng `std::quoted`. Nhờ đó, lý do có dấu cách, dấu nháy hoặc dấu gạch chéo vẫn đọc lại đúng. Một bản dữ liệu minh họa có thể viết như sau:

```
PARKING_DATA 100 3
1 "C001" "29A112345" 1 "2026-10-26 08:00" "2026-10-26 17:00" 3000 0 "" 0 "" "" ""
↪  "" "" "" ""
2 "C002" "29M99999" 2 "2026-10-26 18:00" "-" 0 0 "" 0 "" "" "" "" "" "" ""
3 "C003" "29A577777" 1 "2026-10-27 08:00" "2026-10-27 17:00" 50000 1 "Mất thẻ" 1
↪  "SV001" "Nguyen Van A" "Tran Van B" "" "" "" "Xe bo"
```

Không thể chỉ đọc đủ trường rồi coi dữ liệu đã hợp lệ. Lớp lưu trữ kiểm tra tiêu đề, dữ liệu dư, kiểu xe, ngày giờ, phí, cờ thủ công và thông tin mất thẻ. Sau đó, khi tạo lại bãi, các mã lượt, thẻ và biển số đang gửi được kiểm tra trùng; số xe phải không vượt sức chứa. Phí của lượt cùng ngày cũng được tính lại để đối chiếu với giá trị lưu.

Nếu chưa có tệp chính và chưa có bản sao lưu, chương trình hỏi sức chứa để tạo bãi mới. Nếu có tệp nhưng nội dung sai, chương trình dừng khởi động với thông báo lỗi. Hai trường hợp phải được tách rõ: tệp hỏng không có nghĩa là chưa từng có dữ liệu và không cho phép ghi một bãi trống đè lên lịch sử cũ.



Hình 5.3. Các bước thay thế tệp dữ liệu và vị trí bản sao lưu

Khi lưu, chương trình ghi toàn bộ trạng thái dự kiến vào `.tmp` và kiểm tra luồng sau khi ghi, đẩy dữ liệu, đóng tệp. Nếu có tệp chính, nó được chuyển sang `.bak` trước khi tệp tạm được đưa vào vị trí chính. Khi thay thế thất bại, chương trình cố đưa bản cũ trở lại. Sau khi tệp chính mới đã hoàn tất, lỗi dọn bản sao lưu không làm đảo kết quả giao dịch.

Nếu tiến trình dừng đúng lúc tệp chính đã đổi tên nhưng tệp mới chưa vào vị trí, lần mở sau có thể đọc `.bak` khi thiếu tệp chính. Cách này xử lý được tình huống gián đoạn đã mô phỏng, nhưng không bảo đảm phục hồi mọi lỗi mất điện. Chương trình chỉ dành cho một máy và một phiên làm việc.

5.5 Ngày giờ, tiền và thống kê

`DateTime` lưu năm, tháng, ngày, giờ và phút dưới dạng các giá trị số. Hàm tạo kiểm tra miền giá trị và số ngày của tháng, bao gồm quy tắc năm nhuận. Hàm phân tích chuỗi yêu cầu đúng dạng `YYYY-MM-DD HH:MM`; không chấp nhận một ngày sai rồi tự chuyển sang tháng sau.

Đoạn mã 5.2. Kiểm tra ngày giờ ngay khi tạo đối tượng

```
DateTime::DateTime(int year, int month, int day, int hour, int minute)
    : year_(year), month_(month), day_(day), hour_(hour), minute_(minute) {
    if (year < 1 || year > 9999 || month < 1 || month > 12 || hour < 0 ||
        hour > 23 || minute < 0 || minute > 59) {
        throw ValidationException("Ngày giờ không hợp lệ.");
    }
    const bool leap = (year % 400 == 0) || (year % 4 == 0 && year % 100 !=
    ↪ 0);
    const int days[] = {31, leap ? 29 : 28, 31, 30, 31, 30, 31, 31, 30, 31,
    ↪ 30, 31};
    if (day < 1 || day > days[month - 1]) {
        throw ValidationException("Ngày không tồn tại trong tháng.");
    }
}
```

So sánh thời điểm dựa trên thứ tự năm, tháng, ngày, giờ, phút. Vì dữ liệu trong bài toán là giờ địa phương trên một máy, cách biểu diễn này thuận tiện cho việc so cùng ngày và so với 19:00. Chương trình chưa xử lý việc thay đổi múi giờ hoặc điều chỉnh đồng hồ giữa các cổng. Nếu đồng hồ dẫn đến giờ ra trước giờ vào, thao tác bị từ chối để nhân viên kiểm tra lại.

Số tiền dùng `long long`; các mức phí tự động được trả về bằng số nguyên. Khi hiển thị, giao diện thêm dấu chấm phân cách hàng nghìn, nhưng tệp vẫn lưu số nguyên không có định dạng trang trí. Lý do thu thủ công được giới hạn trên một dòng, tối đa 500 byte UTF-8, để một bản ghi không bị tách thành nhiều dòng. Lượt mất thẻ dùng phí thủ công nên doanh thu vẫn được cộng theo ngày ra như các lượt đã trả khác.

5.6 Build và chạy chương trình

Chương trình dùng C++17 và thư viện chuẩn, đã build trên Windows bằng GCC 16.1.0. Chế độ thường dùng giờ máy; demo cho nhập giờ và dùng dữ liệu riêng. Từ `source_code/`, chạy:

```
powershell -ExecutionPolicy Bypass -File scripts/build.ps1
.\build\parking.exe --demo
powershell -ExecutionPolicy Bypass -File scripts/test.ps1
```

6. Kiểm thử và đánh giá

6.1 Cách chọn tình huống kiểm tra

Kiểm thử được xây dựng quanh những điều có thể làm sai một lượt gửi: chọn nhầm mức phí, nhận trùng xe, kết thúc khi chưa thu tiền hoặc báo thành công dù tệp chưa ghi được. Các ca không chỉ kiểm tra thông báo xuất hiện, mà còn kiểm tra số xe đang gửi, mã lượt kế tiếp, phí và dữ liệu đọc lại sau thao tác.

Bộ kiểm thử C++ dùng `assert` và các hàm hỗ trợ bắt ngoại lệ mong đợi. Mỗi nhóm tình huống có tệp tạm riêng. Các lỗi lưu được mô phỏng ở thư mục kiểm thử, không dùng thư mục dữ liệu thường hoặc demo của người trình diễn. Script PowerShell bổ sung kiểm tra qua chính tệp thực thi để đi qua bước nhập, xác nhận và hiển thị.

Bảng 6.1. Phân nhóm 45 ca kiểm thử C++

Nhóm ca	Nội dung	Số ca
T01–T03	Ngày giờ, chuẩn hóa và loại xe	3
T04–T12	Biểu phí, mốc 19:00, tính bao phủ	9
T13–T24	Nhận/trả, trạng thái, qua đêm, sức chứa, khởi động lại	12
T25–T31	Tệp hỏng, lỗi ghi, phục hồi bản sao lưu	7
T32–T35	Tìm kiếm, phí thủ công, chuỗi tiếng Việt, giới hạn số	4
T36–T45	Mất thẻ và thông tin sinh viên/chủ xe	10
Tổng		45

Kết quả thực chạy là 45 ca C++ đạt và sáu kịch bản console đạt. Đây là kết quả của các tình huống được viết trong bộ kiểm tra, chưa phải một số đo bao phủ mọi dòng lệnh. Các phần dưới chọn những ca liên quan trực tiếp đến quyết định thiết kế để phân tích; danh sách đầy đủ nằm ở Phụ lục A.

6.2 Biên 19:00 và tính đầy đủ của biểu phí

Ba ví dụ thông thường chưa đủ để phát hiện sai dấu so sánh ở mốc 19:00. Nếu quy tắc ban ngày dùng dấu nhỏ hơn hoặc bằng, một xe ra đúng 19:00 có thể bị thu 3.000 thay vì 7.000. Nếu quy tắc buổi tối chỉ nhận giờ vào lớn hơn 19:00, xe vào đúng mốc lại có thể không khớp quy tắc nào.

Bảng 6.2. Các ca ranh giới và số tiền mong đợi

Giờ vào	Giờ ra	Ý nghĩa kiểm tra	Phí
08:00	17:00	Sáng đến chiều cùng một lượt	3.000
18:00	18:59	Trước mốc một phút	3.000
18:00	19:00	Giờ ra đúng mốc	7.000
19:00	21:00	Giờ vào đúng mốc	5.000
19:00	19:00	Hai thời điểm bằng nhau	5.000

Ca T11 kiểm tra rộng hơn: một ngày có 1.440 phút, và với mỗi phút vào, phút ra được duyệt từ chính phút đó đến 23:59. Tổng số cặp hợp lệ là:

$$\frac{1440 \times 1441}{2} = 1.037.520.$$

Với mỗi cặp, phép kiểm tra yêu cầu đúng một quy tắc khớp. Sau đó nó đối chiếu quy tắc đó với vị trí của hai thời điểm so với 19:00. Điều kiện thứ nhất phát hiện chồng lấn hoặc bỏ sót; điều kiện thứ hai tránh trường hợp tuy chỉ khớp một quy tắc nhưng lại là quy tắc sai.

Việc duyệt này có ý nghĩa vì độ chính xác của chương trình chỉ đến phút. Nó không phải kiểm thử tải 1.037.520 xe và không chứng minh tốc độ vận hành tại cổng. Các cặp đều nằm trong một ngày; tính đúng của ngày tháng và xử lý qua ngày được kiểm tra bằng những ca khác.

6.3 Hủy trả xe và sử dụng lại thẻ

Trong T18, một xe vào 08:00 được tra trả lúc 19:00. Báo giá đúng là 7.000 đồng. Sau lời gọi chuẩn bị, bộ kiểm tra đối chiếu số xe vẫn bằng một, doanh thu bằng 0 và nội dung tệp không đổi. Đây là bằng chứng trực tiếp rằng thao tác xem phí không âm thầm đóng lượt.

Ở kịch bản console, lần xem đầu tiên lúc 17:00 hiển thị 3.000 nhưng nhân viên không xác nhận đã thu tiền. Một lần khác hủy ngay tại bước đối chiếu biển số. Chỉ lần xác nhận đầy đủ lúc 19:00 mới đóng lượt với mức 7.000. Kết quả không được suy ra từ dòng thông báo riêng lẻ: script còn đọc tệp và kiểm tra chỉ có đúng một lượt đã trả tương ứng.

T20 kiểm tra thẻ được nhận lại sau khi kết thúc lượt trước. Trong ca này, xe được trả lúc 17:00 và dùng lại thẻ ngay lúc 17:00 để bảo đảm báo giá cũ không chỉ bị từ chối vì

giờ ra trước giờ vào mới. Dịch vụ phát hiện mã lượt đã khác và ném ngoại lệ xung đột. Lượt mới vẫn đang hoạt động; lịch sử của thẻ có hai bản ghi.

Bảng 6.3. Trạng thái mong đợi trong các nhánh trả xe

Thao tác	Xe đang gửi	Đã thu	Thẻ
Chỉ chuẩn bị báo giá	Giữ nguyên	Chưa	Vẫn gắn với lượt
Không xác nhận biển số	Giữ nguyên	Chưa	Vẫn gắn với lượt
Không xác nhận thu tiền	Giữ nguyên	Chưa	Vẫn gắn với lượt
Xác nhận và lưu được	Giảm một	Có	Có thể dùng lại
Lưu thất bại	Giữ nguyên	Chưa ghi nhận	Vẫn gắn với lượt

Các ca này cho thấy lý do phải có mã lượt độc lập, đối tượng báo giá tạm và bước xác nhận riêng. Nếu chỉ kiểm tra ba mức tiền, những sai sót về thời điểm trả thẻ vẫn có thể tồn tại mà không bị phát hiện.

6.4 Mất thẻ và thông tin sinh viên/chủ xe

T36 kiểm tra lượt vào 08:00, ra 17:00 cùng ngày nhưng xử lý mất thẻ: phí tự động 3.000 chỉ để tham khảo, còn phí thực thu do nhân viên nhập là 50.000. Sau khi lưu, lượt đã đóng, doanh thu ghi 50.000 và thông tin sinh viên/chủ xe đọc lại đúng.

Các ca còn lại kiểm tra những điều kiện khác nhau. T37 và T38 từ chối thông tin bắt buộc để trống hoặc biển số không có trong bãi; lượt vẫn đang gửi. T39 và T45 kiểm tra chuỗi tiếng Việt, dấu nháy và gạch chéo. T40 xác nhận thẻ bị mất vẫn dùng lại được, còn T41 chặn báo giá cũ. T42 mô phỏng lỗi ghi để chắc rằng dữ liệu không đổi. T43 và T44 từ chối bản ghi có thông tin mất thẻ nhưng trạng thái không khớp.

6.5 Lỗi ghi tệp và khôi phục dữ liệu

T29 và T30 tạo một thư mục tại đúng đường dẫn dành cho tệp `.tmp`. Khi lớp lưu trữ cố mở đường dẫn đó như tệp để ghi, thao tác thất bại. Cách mô phỏng này ổn định hơn việc trông chờ đĩa đầy hoặc thay quyền hệ thống, đồng thời không tác động đến dữ liệu của người dùng.

Với T29, thao tác nhận xe phải ném `StorageException`; số xe vẫn bằng 0, mã kế tiếp vẫn bằng 1 và tệp chính giữ nguyên từng byte. Với T30, xe đã có sẵn trong bãi; sau khi xác nhận trả nhưng lưu lỗi, số xe vẫn bằng một và doanh thu vẫn bằng 0. Nội dung tệp trước và sau cũng phải giống nhau.

Bảng 6.4. Đối chiếu trước và sau lỗi ghi khi trả xe

Thông tin	Trước xác nhận	Sau lỗi lưu
Số xe đang gửi	1	1
Giờ ra của lượt	Chưa có	Chưa có
Doanh thu trong ngày	0	0
Thẻ đang sử dụng	Có	Có
Nội dung tệp chính	Bản cũ	Không đổi

T31 mô phỏng một điểm gián đoạn khác: đã có dữ liệu hợp lệ, tệp chính được đổi tên sang `.bak`, rồi khởi động dịch vụ khi tệp chính không còn. Chương trình phải đọc được số xe và sức chứa từ bản sao lưu. Sau một thay đổi sức chứa hợp lệ, tệp chính mới xuất hiện và vẫn giữ lượt đang gửi.

Các ca tệp hỏng kiểm tra tiêu đề, phí cùng ngày, thẻ trùng, ngày sai và mã kế tiếp không hợp lệ. Ca T25 còn so sánh nội dung tệp trước và sau khởi động thất bại. Vì vậy, việc không ghi đè tệp hỏng được kiểm tra bằng dữ liệu thực, không chỉ dựa vào thông báo lỗi.

Nhóm kiểm thử này chưa mô phỏng mọi điểm lỗi của hệ điều hành hoặc mất điện trong từng bước đổi tên. Nó chỉ xác nhận cách xử lý tại những điểm đã được thiết kế trong bộ kiểm tra.

6.6 Kết quả chạy qua giao diện console

Các kịch bản đầu cuối chạy đúng tệp thực thi `parking.exe`, đưa dữ liệu qua luồng nhập và kiểm tra cả đầu ra lẫn tệp tạo ra. Nhờ vậy, phần đọc menu và hai bước xác nhận được kiểm tra cùng với dịch vụ, thay vì chỉ gọi phương thức C++ trực tiếp.

Bảng 6.5. Sáu kịch bản console đã chạy

Mã	Nội dung	Kết quả
C01	Nhập menu sai, hủy trả, dùng lại thẻ, qua đêm và thống kê	Đạt
C02	Mở lại, xem xe đang gửi và lịch sử; tệp không bị sửa	Đạt
C03	Chế độ thường lấy đúng ngày hệ thống để nhận xe	Đạt
C04	Tệp hỏng làm dừng khởi động và không bị ghi đè	Đạt
C05	Mất thẻ trong ngày, lưu thông tin sinh viên/chủ xe, phí thủ công	Đạt
C06	Thẻ bị mất vẫn dùng lại được	Đạt

Kịch bản C01 kết thúc với bốn lượt: lượt thứ nhất thu 7.000, lượt thứ hai dùng lại thẻ và thu 5.000, lượt thứ ba qua ngày thu 12.000, lượt thứ tư còn đang gửi. Thời gian do chế độ demo cung cấp để kiểm tra từng nhánh, không phải dữ liệu xếp hàng tại bãi xe thực tế. Đầu ra dưới đây được trích từ lần chạy đó và chỉ bỏ phần menu lặp lại:

Đoạn mã 6.1. Một số dòng kết quả từ kịch bản C01

```
SỐ TIỀN CẦN THU: 3.000 đồng
Chưa đóng lượt gửi; xe vẫn nằm trong danh sách đang gửi.
Đã hủy trả xe. Lượt gửi vẫn đang hoạt động.
SỐ TIỀN CẦN THU: 7.000 đồng
SỐ TIỀN CẦN THU: 5.000 đồng
SỐ TIỀN CẦN THU: 12.000 đồng
Ngày 2026-10-26 | Lượt vào: 3 | Lượt ra: 2 | Doanh thu thực thu: 12.000 đồng
Ngày 2026-10-27 | Lượt vào: 0 | Lượt ra: 1 | Doanh thu thực thu: 12.000 đồng
```

Ngày 26/10 có ba lượt vào, hai lượt ra và 12.000 đồng thực thu. Ngày 27/10 có một lượt ra qua đêm với 12.000 đồng. Dòng báo giá 3.000 không được cộng vào doanh thu vì lần đó bị hủy. Lượt thứ tư được nhận sau hai lần thống kê, nên bảng tổng hợp khi ấy chưa có lượt vào mới của ngày 27. Chi tiết thứ tự này cần được giữ khi đối chiếu ảnh hoặc nhật ký với dữ liệu cuối kịch bản.

C05 dùng thư mục riêng để nhận C010, xử lý mất thẻ trong ngày với phí 50.000 và kiểm tra thông tin sinh viên/chủ xe trong tệp. C06 nhận lại bằng C010 để xác nhận thẻ không bị khóa.

6.7 Đánh giá thiết kế và giới hạn sử dụng

Kết quả kiểm thử xác nhận các ràng buộc đã chọn trong phạm vi một phiên. Cách lưu hiện tại duyệt và ghi lại toàn bộ lịch sử sau mỗi thay đổi, nên phù hợp với lượng dữ liệu nhỏ của bài tập. Khi lịch sử tăng nhiều, cần tách nhu cầu tra xe hiện tại khỏi nhu cầu lưu lịch sử dài hạn.

Chương trình chưa có đăng nhập, phân quyền, kết nối phần cứng hay xử lý nhiều cổng. Việc đối chiếu biển số vẫn phụ thuộc nhân viên. Phần mất thẻ chỉ mô phỏng giấy tờ bằng nhập tay, không lưu ảnh và không khóa thẻ.

7. Kết luận và hướng phát triển

7.1 Mức đáp ứng mục tiêu

Đề tài đã mô hình hóa được một lượt gửi xe từ lúc nhận thẻ đến lúc thu tiền và trả thẻ. Kết quả chính nằm ở sự liên kết giữa những dữ liệu dùng chung: một thẻ chỉ thuộc một lượt đang gửi, phí gắn với thời gian của chính lượt đó, lịch sử không mất khi thẻ được dùng lại và xe chưa ra vẫn được khôi phục sau khi mở lại chương trình.

Năm nội dung trọng tâm của học phần đều có phần cài đặt cụ thể. Yêu cầu được chuyển thành điều kiện trước và sau thao tác. Các lớp xe, lượt gửi, bãi và thời gian giữ những trách nhiệm khác nhau. Kế thừa và đa hình tập trung ở quy tắc phí; nhóm ngoại lệ phân biệt các nguyên nhân thất bại. Những đoạn mã trong báo cáo được lấy từ chương trình để việc giải thích có thể đối chiếu trực tiếp khi bảo vệ bài.

Kết quả 45 ca C++ và sáu kịch bản console hỗ trợ các kết luận trong phạm vi đã thử. Đặc biệt, các ca hủy trả, báo giá cũ và lỗi ghi tệp kiểm tra được thời điểm dữ liệu được phép thay đổi. Chúng bổ sung cho kiểm thử mức tiền, vì một mức phí đúng vẫn chưa đủ nếu lượt bị kết thúc nhầm hoặc doanh thu được ghi trước khi xác nhận. Luồng mất thẻ bổ sung một đường tra cứu theo biển số, lưu thông tin sinh viên/chủ xe và thu phí thủ công; các ca T36–T45 xác nhận lượt vẫn đóng đúng và dữ liệu đọc lại đầy đủ.

7.2 Những lựa chọn cần cân nhắc khi mở rộng

Cách lưu bằng tệp văn bản giúp nhóm dễ theo dõi giao dịch và tạo lại các tình huống lỗi, nhưng toàn bộ lịch sử phải được duyệt và ghi lại sau mỗi thay đổi. Nếu dùng với lượng dữ liệu lớn hơn, cần thay lớp lưu trữ và cách tra cứu trước khi nghĩ đến việc đổi giao diện.

Quy tắc qua đêm cũng cần được thống nhất trước khi tự động hóa. Đề bài mới chốt ba mức phí trong cùng ngày, nên chương trình yêu cầu nhân viên nhập phí và lý do cho lượt qua ngày. Một công thức tự động chỉ nên được thêm khi chính sách đã rõ.

7.3 Hướng phát triển

Trước mắt, nhóm cần thống nhất quy tắc qua đêm và cách xử lý khi lưu thất bại. Sau đó có thể bổ sung kho thẻ, tài khoản nhân viên và nhật ký thao tác. Khi quy mô tăng, lớp lưu trữ có thể chuyển sang cơ sở dữ liệu và hỗ trợ nhiều cổng; phần cứng như đầu đọc thẻ hay camera chỉ nên thêm sau khi quy trình nghiệp vụ đã ổn định.

Kiến trúc tách giao diện, dịch vụ và lưu trữ tạo điểm bắt đầu thuận tiện cho các thay đổi này, nhưng mỗi bước vẫn phải được kiểm thử lại. Trong phạm vi học phần, sản phẩm hiện tại đủ để trình diễn một quy trình có dữ liệu, có điều kiện lỗi và có các cơ chế OOP có thể giải thích rõ ràng.

A. Ma trận kiểm thử và dữ liệu minh họa

A.1 Kết quả kiểm thử C++

Các mã dưới đây tương ứng với tên ca trong chương trình kiểm thử. Mỗi dòng ghi điều cần kiểm tra; cột kết quả phản ánh lần chạy của bộ mã bàn giao. Các ca đầu cuối qua console được trình bày trong phần chính.

Bảng A.1. Ma trận đầy đủ của 45 ca kiểm thử

Mã	Nội dung và kết quả mong đợi	Kết quả
T01	Chấp nhận ngày hợp lệ và năm nhuận 2000, 2024.	Đạt
T02	Từ chối ngày không tồn tại, giờ và định dạng sai.	Đạt
T03	Chuẩn hóa thẻ/biển số; từ chối loại xe không hỗ trợ.	Đạt
T04	Vào 08:00, ra 17:00: phí 3.000 đồng.	Đạt
T05	Vào 18:00, ra 18:59: phí 3.000 đồng.	Đạt
T06	Vào 18:00, ra đúng 19:00: phí 7.000 đồng.	Đạt
T07	Vào đúng 19:00, ra 21:00: phí 5.000 đồng.	Đạt
T08	Vào 08:00, ra 21:00: phí 7.000 đồng.	Đạt
T09	Vào và ra cùng phút vẫn tính phí theo khung.	Đạt
T10	Không tự tính phí qua ngày; từ chối giờ ra trước giờ vào.	Đạt
T11	Mỗi cặp phút cùng ngày hợp lệ khớp đúng một quy tắc phí.	Đạt
T12	Hai loại xe cùng thời gian có cùng mức phí.	Đạt
T13	Nhận xe lưu dữ liệu và tăng mã lượt kế tiếp.	Đạt
T14	Thẻ đang dùng bị từ chối, không tiêu hao mã lượt.	Đạt
T15	Biển số trùng sau chuẩn hóa bị từ chối.	Đạt
T16	Bãi đầy không nhận thêm xe.	Đạt
T17	Báo lỗi thẻ không có lượt gửi, giờ ra không hợp lệ.	Đạt
T18	Xem báo giá không đóng lượt, thu tiền hoặc ghi tệp.	Đạt
T19	Xác nhận tính lại phí, ghi doanh thu, chặn trả lặp.	Đạt
T20	Dùng lại thẻ với mã lượt mới, chặn báo giá cũ.	Đạt
T21	Qua đêm bắt buộc có phí không âm và lý do.	Đạt
T22	Phí qua đêm ghi vào ngày ra, kể cả khi đổi tháng.	Đạt
T23	Sức chứa mới không dưới số xe; thay đổi hợp lệ được lưu.	Đạt

Mã	Nội dung và kết quả mong đợi	Kết quả
T24	Mở lại giữ xe đang gửi, lịch sử, sức chứa và mã kế tiếp.	Đạt
T25	Tiêu đề tệp hỏng bị từ chối, nội dung không bị ghi đè.	Đạt
T26	Từ chối tệp có phí cùng ngày sai biểu phí.	Đạt
T27	Từ chối hai lượt đang gửi dùng chung thẻ trong tệp.	Đạt
T28	Từ chối ngày sai và mã lượt kế tiếp không hợp lệ.	Đạt
T29	Lỗi ghi khi nhận xe: bộ nhớ và tệp cũ không đổi.	Đạt
T30	Lỗi ghi khi trả xe: xe còn trong bãi, doanh thu chưa có.	Đạt
T31	Thiếu tệp chính: đọc bản sao lưu và lưu lại được.	Đạt
T32	Tìm kiếm đã chuẩn hóa trả đủ các lượt lịch sử.	Đạt
T33	Không cho sửa phí thủ công của lượt cùng ngày.	Đạt
T34	Giữ đúng lý do tiếng Việt có dấu nháy/gạch chéo và phí 0.	Đạt
T35	Từ chối ngày thống kê sai; phát hiện tổng tiền tràn số.	Đạt
T36	Mất thẻ trong ngày: phí thủ công, doanh thu và thông tin được lưu.	Đạt
T37	Thiếu mã SV/CCCD, họ tên SV hoặc họ tên chủ xe bị từ chối.	Đạt
T38	Biển số không có lượt đang gửi bị từ chối.	Đạt
T39	Thông tin mất thẻ có tiếng Việt, trường tùy chọn để trống.	Đạt
T40	Thẻ bị mất vẫn được dùng lại cho lượt mới.	Đạt
T41	Báo giá mất thẻ cũ bị từ chối sau khi lượt đã đóng.	Đạt
T42	Lỗi ghi khi xử lý mất thẻ: bộ nhớ và tệp cũ không đổi.	Đạt
T43	Từ chối bản ghi có thông tin mất thẻ nhưng lượt đang gửi.	Đạt
T44	Từ chối bản ghi không mất thẻ nhưng có trường mất thẻ.	Đạt
T45	Giữ đúng mã SV, họ tên và ghi chú có dấu nháy/gạch chéo.	Đạt

A.2 Dữ liệu kết thúc kịch bản console

Bảng sau tổng hợp bốn lượt của C01. Thời gian theo ngày 26 và 27/10/2026. Hai lần thống kê trong nhật ký diễn ra trước khi nhận lượt thứ tư.

Bảng A.2. Các lượt được tạo trong kịch bản C01

Lượt	Thẻ	Thời gian vào	Thời gian ra	Phí
1	C001	26/10, 08:00	26/10, 19:00	7.000
2	C001	26/10, 19:00	26/10, 21:00	5.000
3	C002	26/10, 18:00	27/10, 08:00	12.000
4	C003	27/10, 08:00	Chưa ra	0

C05 dùng một thư mục riêng: nhận C010, xử lý mất thẻ trong ngày với phí 50.000, lưu mã SV/CCCD, họ tên sinh viên và họ tên chủ xe. C06 nhận lại bằng C010 để xác nhận thẻ không bị khóa.

Để chạy lại toàn bộ kiểm tra trên Windows, dùng script `source_code/scripts/test.ps1`. Nhật ký C++ và console được tạo trong thư mục build. Các thư mục kiểm thử tạm được tạo riêng và dọn sau khi chạy; chúng không thay thế dữ liệu ở `data/` hoặc `demo_data/`.